ETHICAL HACKING & FORENSICS

THE PRACTICAL MASTER HANDBOOK

Twenty questions. Nine experiments.
One clear, exam-ready playbook.

BUILT FOR PRACTICAL REVISION

Command → Expected output → Why it matters
Source-mapped procedures + interpretation + exam tips
Offline companion: interactive HTML study app

REFERENCE EDITION · OCTOBER 2026

READ THIS FIRST / 01

A practical-first revision system.

This handbook consolidates all 20 questions in the supplied EHF Lab Practice Questions paper, mapping each one to Experiments 1-8 and 10. Every solution is built as a short execution path: command or GUI action, expected evidence, and purpose.

01 / EXECUTE

Use the commands in sequence. Commands are designed as exam-ready examples, not as unverified output from your current machine.

02 / OBSERVE

Compare what appears on your own screen with the illustrative outputs. Record actual target IPs, filenames, timestamps, offsets and hashes.

03 / INTERPRET

Conclude each exercise with the observation. Distinguish an open service from an established vulnerability, and a string match from proven activity.

04 / VERIFY

Use assigned authorized domains/hosts only. Preserve forensic originals; work on evidence copies, and never assume a missing artifact exists.

WHAT COMES FROM THE LAB

Experiment mapping, demonstrated tools and observations are taken from the attached EHF records. Small variations in filenames, sample content and GUI choices are explicitly adapted when a practice question asks for a different target or source type.

TWO IMPORTANT DISTINCTIONS

Experiment 4 creates a .dd copy of a TAR archive rather than an entire partitioned disk. Experiment 7 examines a raw disk image, but its reported acquisition was incomplete; do not describe missing-data errors as successful recovery.

YOUR EXAM ROADMAP / 02

Question index

Every question is indexed by topic and its originating experiment. Use the question number in the margin when revising.

01	Create, copy and rename evidence	EXP 01
02	Domain registration and web technologies	EXP 02
03	File management, current user and hostname	EXP 01
04	Open-port and service scan	EXP 03
05	Kali system details and IP address	EXP 01 + 03
06	Create a logical forensic image and verify hashes	EXP 04
07	Collect public emails, hosts and DNS	EXP 02
08	Autopsy case creation and two artifacts	EXP 05
09	Combined host and website reconnaissance	EXP 02
10	Partition and file-system identification	EXP 07
11	Nmap service and software-version detection	EXP 03
12	Extract email and URL artifacts	EXP 06
13	Classify TCP, UDP, DNS and ICMP traffic	EXP 08
14	Autopsy metadata, deleted files and keyword hits	EXP 05
15	Investigate email origin and key headers	EXP 10
16	List image files, inspect inodes and extract	EXP 07
17	Trace email routing and authentication	EXP 10
18	Create, rename, display and delete files	EXP 01
19	Analyze DNS exchange or TCP handshake	EXP 08
20	Extract emails, URLs and IP artifacts	EXP 06

RECOMMENDED REVISION ORDER

Linux basics → reconnaissance → Nmap → imaging → Autopsy → Bulk Extractor → Sleuth Kit → Wireshark → email forensics. This order groups repeated commands and reduces memorization time.

QUESTION 01 / EXPERIMENT 01 / LINUX ESSENTIALS / Exp 1, pp. 2-5

Create, copy and rename evidence

Create a directory named EHF_Practical and a file named evidence.txt. Write and display its contents, create a copy, rename the copied file, and verify the final directory contents using appropriate Linux commands.

TARGET OUTCOME

Produce two identical files in EHF_Practical: the original evidence.txt and renamed evidence_backup.txt.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
mkdir EHF_Practical
```

EXPECT No printed output when successful.

PURPOSE Creates the required directory.

02 TERMINAL

```
cd EHF_Practical
```

EXPECT Shell location changes to EHF_Practical.

PURPOSE All subsequent file commands operate here.

03 TERMINAL

```
echo "EHF Practical Evidence" > evidence.txt
```

EXPECT No printed output; text is written to evidence.txt.

PURPOSE Creates the file and writes a reproducible sample line.

04 TERMINAL

```
cat evidence.txt
```

EXPECT EHF Practical Evidence

PURPOSE Confirms the content was stored.

05 TERMINAL

```
cp evidence.txt evidence_copy.txt
```

EXPECT No output; second file appears.

PURPOSE Duplicates the original without changing it.

06 TERMINAL

```
mv evidence_copy.txt evidence_backup.txt
```

EXPECT No output; copy is renamed.

PURPOSE Renames the duplicate.

07 TERMINAL

```
ls -l
```

EXPECT Lists evidence.txt and evidence_backup.txt, with metadata.

PURPOSE Verifies final files and their sizes.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ cat evidence.txt
EHF Practical Evidence
$ ls
evidence.txt evidence_backup.txt
```

HOW TO INTERPRET THE EVIDENCE

Both filenames must appear in the final listing; evidence_copy.txt should not remain. File permissions and dates depend on the machine.

WRITE THIS AS YOUR FINAL OBSERVATION

The evidence directory, original text file, and renamed copy were created and verified successfully.

EXAM TIP

Run commands in order. The > operator replaces existing content; use a new working directory to avoid overwriting evidence.

QUESTION 02 / EXPERIMENT 02 / RECONNAISSANCE / Exp 2, pp. 2, 9-10

Domain registration and web technologies

Perform reconnaissance on an authorized domain to obtain publicly available domain registration details and identify the technologies used by its website. Record the observations and explain the significance of the information collected.

TARGET OUTCOME

Retrieve public registration details and fingerprints of the site technology stack.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
whois cloudflare.com
```

EXPECT Domain Name, Registrar, Creation Date, Registry Expiry Date, Name Servers, Domain Status (fields vary).

PURPOSE Shows public registration and administrative records.

02 TERMINAL

```
whatweb cloudflare.com
```

EXPECT URL, HTTP status, redirects, server/header/cookie or framework fingerprints where detectable.

PURPOSE Fingerprints observable web technologies.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ whois cloudflare.com
Domain Name: CLOUDFLARE.COM
Registrar: ...
Name Server: ...
$ whatweb cloudflare.com
https://cloudflare.com [200 OK] ...
```

HOW TO INTERPRET THE EVIDENCE

WHOIS reveals registrar and DNS delegation; WhatWeb reports technologies inferred from HTTP responses. Redacted registration fields and incomplete fingerprints are normal.

WRITE THIS AS YOUR FINAL OBSERVATION

Public registration and website information were collected to understand domain ownership context, DNS infrastructure, and externally visible technologies.

EXAM TIP

Use the assigned authorized domain in place of cloudflare.com. A technology fingerprint is not proof of a security vulnerability.

QUESTION 03 / EXPERIMENT 01 / LINUX ESSENTIALS / Exp 1, pp. 1-8

File management, current user and hostname

Demonstrate basic file and directory management in Kali Linux. Create a directory, create a file inside it, navigate between directories, remove a file, and verify the results. Also display the current user and system hostname.

TARGET OUTCOME

Create and remove a sample file, verify its absence, and identify user and host.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
pwd
```

EXPECT /home/student (illustrative).

PURPOSE Reports the starting directory.

02 TERMINAL

```
mkdir EHF_Lab
```

EXPECT No output.

PURPOSE Makes a new workspace.

03 TERMINAL

```
cd EHF_Lab
```

EXPECT Working directory changes.

PURPOSE Enters the workspace.

04 TERMINAL

```
touch sample.txt
```

EXPECT No output.

PURPOSE Creates an empty sample file.

05 TERMINAL

```
ls
```

EXPECT sample.txt

PURPOSE Verifies creation.

06 TERMINAL

```
rm sample.txt
```

EXPECT No output.

PURPOSE Deletes the sample file.

07 TERMINAL

```
ls -la
```

EXPECT Shows directory entries but not sample.txt.

PURPOSE Proves the file was removed.

08 TERMINAL

```
cd ..
```

EXPECT Returns to the parent directory.

PURPOSE Demonstrates navigation.

09 TERMINAL

```
whoami
```

EXPECT student (example).

PURPOSE Displays active username.

10 TERMINAL

```
hostname
```

EXPECT kali (example).

PURPOSE Displays machine hostname.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ ls
sample.txt
$ rm sample.txt
$ ls -la
total ...
. ..
$ whoami
student
$ hostname
kali
```

HOW TO INTERPRET THE EVIDENCE

The file should appear before rm and disappear afterward. Username and hostname must be recorded from the actual Kali VM.

WRITE THIS AS YOUR FINAL OBSERVATION

Directory/file operations, navigation, file removal, username identification, and hostname identification were completed.

EXAM TIP

rm is permanent under a normal terminal workflow; check the name before deleting.

QUESTION 04 / EXPERIMENT 03 / NETWORK SCANNING / Exp 3, pp. 2-3

Open-port and service scan

Scan an assigned lab machine to identify open ports and running services. Interpret the scan output and identify potential security concerns.

TARGET OUTCOME

Identify the reachable TCP services exposed by the authorized lab machine.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
nmap LAB_IP
```

EXPECT Lists detected PORT, STATE and SERVICE columns.

PURPOSE Basic TCP port scan.

02 TERMINAL

```
nmap -sV LAB_IP
```

EXPECT Lists open ports and attempts software/service fingerprinting.

PURPOSE Adds useful evidence for interpreting exposure.

ILLUSTRATIVE CONSOLE / FINDINGS

```
PORT STATE SERVICE
22/tcp open ssh
80/tcp open http
110/tcp open pop3
143/tcp open imap
443/tcp open https
```

HOW TO INTERPRET THE EVIDENCE

Ports 22/SSH expose remote access, 80/HTTP may transmit plaintext, 110/POP3 and 143/IMAP may use plaintext unless TLS is negotiated, and 443/HTTPS depends on TLS configuration. Open ports alone are not vulnerabilities.

WRITE THIS AS YOUR FINAL OBSERVATION

Nmap identified accessible ports and their associated services. The observed exposure supports a preliminary security review.

EXAM TIP

Replace LAB_IP with the assigned authorized target. The example port list was observed in Exp 3 and is not promised for a different machine.

QUESTION 05 / EXPERIMENT 01 + 03 / LINUX ESSENTIALS / Exp 1, pp. 6-8; Exp 3, p. 1

Kali system details and IP address

Use basic Linux commands to display the operating system information, current date, command history, and network interface details. Record the operating system name and IP address of the Kali Linux machine.

TARGET OUTCOME

Record distribution name and the IP on the active interface.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
uname -a
```

EXPECT Kernel release, architecture, and hostname.

PURPOSE Shows low-level system information.

02 TERMINAL

```
cat /etc/os-release
```

EXPECT PRETTY_NAME="Kali GNU/Linux Rolling" (example).

PURPOSE Obtains the Linux distribution label.

03 TERMINAL

```
date
```

EXPECT Current system date, time, and timezone.

PURPOSE Checks the machine clock.

04 TERMINAL

```
history
```

EXPECT Available prior shell commands, numbered.

PURPOSE Displays shell command history.

05 TERMINAL

```
ip a
```

EXPECT Network interfaces (e.g., lo and eth0), flags, and inet addresses.

PURPOSE Finds the IP address of the connected adapter.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
$ ip a
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
inet 10.0.2.15/24 ...
```

HOW TO INTERPRET THE EVIDENCE

Use the address on the active eth0/ens* interface, not the loopback 127.0.0.1 address. The IP 10.0.2.15 appears in the experimental VM as an example.

WRITE THIS AS YOUR FINAL OBSERVATION

Kali Linux operating-system information, system date, shell history, network-interface details, and the assigned interface IP were displayed.

EXAM TIP

Write down the actual PRETTY_NAME and IPv4 address in the record; they are machine-specific.

QUESTION 06 / EXPERIMENT 04 / FORENSIC IMAGING / Exp 4, pp. 1-3

Create a logical forensic image and verify hashes

Create a forensic image of a logical file. Verify the image size, calculate the cryptographic hashes of the original file and its image, compare the results, and explain the importance of evidence integrity.

TARGET OUTCOME

Create a TAR-based logical source, duplicate it with dd, and check both SHA-256 hashes.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
mkdir -p sample_data forensic_evidence
```

EXPECT Directories exist.

PURPOSE Prepares evidence and destination locations.

02 TERMINAL

```
echo "Digital Forensics Lab" > sample_data/file1.txt
```

EXPECT A sample file is created.

PURPOSE Creates reproducible logical source evidence.

03 TERMINAL

```
tar -cvf sample_data.tar sample_data
```

EXPECT Lists sample_data/ and sample_data/file1.txt.

PURPOSE Packages the directory as an archive.

04 TERMINAL

```
dd if=sample_data.tar of=forensic_evidence/evidence.dd bs=4M status=progress
```

EXPECT Records in/out and copied byte count.

PURPOSE Makes an exact byte copy of the archive.

05 TERMINAL

```
ls -lh sample_data.tar forensic_evidence/evidence.dd
```

EXPECT Shows both file sizes.

PURPOSE Checks the image exists and size matches.

06 TERMINAL

```
sha256sum sample_data.tar forensic_evidence/evidence.dd
```

EXPECT Two 64-character hex digests.

PURPOSE Compares original and image bytes cryptographically.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ sha256sum sample_data.tar forensic_evidence/evidence.dd
<sustained 64-character hash> sample_data.tar
<same 64-character hash> forensic_evidence/evidence.dd
```

HOW TO INTERPRET THE EVIDENCE

Matching SHA-256 digests are strong evidence that the copy matches the original at verification time; a differing digest indicates a mismatch. The .dd here holds TAR data, not a partitioned disk.

WRITE THIS AS YOUR FINAL OBSERVATION

A logical evidence archive was copied, its size checked, and the original and image hashes compared to verify integrity.

EXAM TIP

Do not point dd at a real disk/device in this task. Compare the original file to its copy, not only a single hash of the image.

QUESTION 07 / EXPERIMENT 02 / RECONNAISSANCE / Exp 2, pp. 3-6

Collect public emails, hosts and DNS

Perform reconnaissance on an authorized domain to collect publicly available email addresses, subdomains, and DNS information. Record your findings and explain the significance of any two observations.

TARGET OUTCOME

Look for publicly indexed host/email features and DNS records.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
theHarvester -d cloudflare.com -b urlscan
```

EXPECT Publicly indexed ASNs, interesting URLs, IPs and possible host/email results.

PURPOSE Searches the Urlscan source used in the experiment.

02 TERMINAL

```
dnsenum cloudflare.com
```

EXPECT A/NS/MX records, discoverable hosts, and attempted checks (results vary).

PURPOSE Enumerates available DNS information.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ theHarvester -d cloudflare.com -b urlscan
[*] Target: cloudflare.com
[*] Searching Urlscan
[*] ASNs found: 11
[*] Interesting URLs found: 4
[*] IPs found: 32
$ dnsenum cloudflare.com
Host addresses / Name servers / Mail servers / ...
```

HOW TO INTERPRET THE EVIDENCE

Subdomains suggest additional exposed services; MX records indicate the mail-handling infrastructure. The counts above belong to the Exp 2 screenshot and are not live counts. The source may return no emails.

WRITE THIS AS YOUR FINAL OBSERVATION

Public domain features and DNS records were examined. Subdomains and mail servers provide clues about service segmentation and email infrastructure.

QUESTION 08 / EXPERIMENT 05 / AUTOPSY GUI / Exp 5, pp. 1-7

Autopsy case creation and two artifacts

Create a forensic case using Autopsy and analyze a disk image. Examine available files and metadata, identify and interpret at least two relevant digital artifacts.

TARGET OUTCOME

Create a case, load the assigned disk image, and report two evidence-backed artifacts.

COMMAND-BY-COMMAND PLAYBOOK

01 AUTOPSY / WIRESHARK

```
Autopsy → New Case
```

EXPECT New Case wizard appears.

PURPOSE Begins a forensic case.

02 AUTOPSY / WIRESHARK

```
Case name: EHF_Investigation → choose base directory → Finish
```

EXPECT Case workspace is created.

PURPOSE Records identifying case information.

03 AUTOPSY / WIRESHARK

```
Add Data Source → Disk Image or VM File
```

EXPECT Image-source selector appears.

PURPOSE Uses disk-image mode required by Q8.

04 AUTOPSY / WIRESHARK

```
Select supplied .dd/.img/.E01 image
```

EXPECT Image is attached to the case.

PURPOSE Makes evidence available for ingest.

05 AUTOPSY / WIRESHARK

```
Enable File Type Identification and Keyword Search → ingest
```

EXPECT Results and data-source tree populate.

PURPOSE Indexes relevant content and metadata.

06 AUTOPSY / WIRESHARK

```
Data Sources → select file → File Metadata
```

EXPECT Path, size, allocated status and any timestamps.

PURPOSE Examines first relevant artifact.

07 AUTOPSY / WIRESHARK

```
Deleted Files or Keyword Hits → select item
```

EXPECT Artifact previews, metadata, or hits appear.

PURPOSE Examines a second relevant artifact.

ILLUSTRATIVE CONSOLE / FINDINGS

```
Artifact 1 (Exp 5 example):
f0000000_Microsoft_Word_EHF_EXP_1_1.pdf
MIME: application/pdf | Allocation: unallocated
Artifact 2 (Exp 5 example):
Keyword "user" → 3,572 matches
```

HOW TO INTERPRET THE EVIDENCE

Artifact examples above were reported in Exp 5. They are illustrative and must be replaced by two artifacts actually present in the supplied image, along with paths/timestamps as available.

WRITE THIS AS YOUR FINAL OBSERVATION

An Autopsy case was created and the supplied image examined. Two identifiable artifacts were documented with metadata and their investigative relevance.

EXAM TIP

Exp 5 originally uses Logical Files from C:\Users; Q8 explicitly requests a disk image, so choose Disk Image or VM File for this question.

QUESTION 09 / EXPERIMENT 02 / RECONNAISSANCE / Exp 2, pp. 2-11

Combined host and website reconnaissance

Gather publicly available information about an authorized domain or website, including domain details, host information, and web technologies. Interpret the information collected.

TARGET OUTCOME

Build a compact profile using WHOIS, Dmitry, DNSenum and WhatWeb.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
whois cloudflare.com
```

EXPECT Registrar, registration dates, NS entries, status where public.

PURPOSE Domain registration context.

02 TERMINAL

```
dmitry -wise cloudflare.com
```

EXPECT Host IP, WHOIS and related host/subdomain observations as available.

PURPOSE Combines several information gathering functions from Exp 2.

03 TERMINAL

```
dnsenum cloudflare.com
```

EXPECT Host DNS records, mail servers and found subdomains.

PURPOSE Adds DNS-based infrastructure details.

04 TERMINAL

```
whatweb cloudflare.com
```

EXPECT HTTP status, headers, server/framework fingerprints if detectable.

PURPOSE Identifies website technology indicators.

ILLUSTRATIVE CONSOLE / FINDINGS

```
WHOIS → registrar / creation / name servers
Dmitry → host, IP and related records
DNSenum → A / NS / MX / candidate subdomains
WhatWeb → HTTP status / server / technologies
```

HOW TO INTERPRET THE EVIDENCE

The tools provide complementary perspectives. Some overlap and missing fields are expected; do not equate reconnaissance output with confirmed vulnerabilities.

WRITE THIS AS YOUR FINAL OBSERVATION

Public registration, host-related, DNS and web technology information was collected and interpreted for the authorized website.

EXAM TIP

Dmitry flags vary by installed version; run dmitry -h if the specific lab version rejects an option (help is not an evidence step).

QUESTION 10 / EXPERIMENT 07 / THE SLEUTH KIT / Exp 7, pp. 4-6

Partition and file-system identification

Examine a supplied forensic disk image using The Sleuth Kit. Identify the partition layout, file system type, and relevant file system information wherever available.

TARGET OUTCOME

List image partition offsets, then analyze the selected partition file system.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
mmls -V
```

EXPECT The Sleuth Kit version information.

PURPOSE Confirms availability.

02 TERMINAL

```
mmls evidence/data.dd
```

EXPECT DOS/MBR or other partition table with start/end sectors.

PURPOSE Identifies partition layout.

03 TERMINAL

```
fsstat -o 2048 evidence/data.dd
```

EXPECT File-system type, inode layout, block size and other metadata.

PURPOSE Examines a partition that starts at sector 2048.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ mmls evidence/data.dd
... DOS Partition Table ...
... Linux ... (start: 2048, in Exp 7)
... Extended ...
$ fsstat -o 2048 evidence/data.dd
File System Type: Ext4
Block Size: 4096
...
```

HOW TO INTERPRET THE EVIDENCE

Exp 7 reported an MBR layout with Linux and extended partitions and an Ext4 file system. fsstat offsets are sector counts, not byte offsets.

WRITE THIS AS YOUR FINAL OBSERVATION

The Sleuth Kit revealed partition offsets and file-system structure, including the available Ext4 metadata.

EXAM TIP

2048 applies to the Linux partition in the example; substitute the real start sector from mmls. Exp 7 acquisition was truncated, so missing-data errors may be legitimate.

QUESTION 11 / EXPERIMENT 03 / NETWORK SCANNING / Exp 3, p. 2

Nmap service and software-version detection

Perform service and version detection on an authorized lab target. Identify open ports and associated services, interpret the results, and document potential security concerns.

TARGET OUTCOME

Identify ports and attempt service-version fingerprinting in one scan.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
nmap -sV LAB_IP
```

EXPECT PORT / STATE / SERVICE / VERSION rows for detected open ports.

PURPOSE Performs basic discovery plus service-version attempts.

ILLUSTRATIVE CONSOLE / FINDINGS

```
PORT STATE SERVICE VERSION
22/tcp open ssh protocol 2.0
80/tcp open http ...
110/tcp open pop3 ...
143/tcp open imap ...
443/tcp open ssl/https ...
```

HOW TO INTERPRET THE EVIDENCE

An identified product version can be checked for known patches and configuration issues. SSH protocol 2.0 is a protocol specification, not the installed OpenSSH software release.

WRITE THIS AS YOUR FINAL OBSERVATION

Open services and discoverable version indicators were documented; potential exposure was discussed without asserting a confirmed exploit.

EXAM TIP

Only scan authorized targets. Unknown product versions should be recorded as unresolved.

QUESTION 12 / EXPERIMENT 06 / BULK EXTRACTOR / Exp 6, pp. 2-4

Extract email and URL artifacts

Analyze a forensic disk image to extract network-related artifacts, including available email addresses and URLs. Examine the generated output files and record the relevant findings.

TARGET OUTCOME

Run Bulk Extractor and inspect its email and URL feature files.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
ls -lh forensic_evidence/evidence.dd
```

EXPECT Image path and size.

PURPOSE Confirms input evidence exists.

02 TERMINAL

```
bulk_extractor -o bulk_output forensic_evidence/evidence.dd
```

EXPECT Scanning progress, output location and summary.

PURPOSE Processes the supplied image.

03 TERMINAL

```
ls bulk_output
```

EXPECT Feature files such as email.txt, url.txt, ip.txt and report.xml (set varies).

PURPOSE Locates analysis results.

04 TERMINAL

```
cat bulk_output/email.txt
```

EXPECT Offsets, extracted email features, context; may have no matches.

PURPOSE Examines email artifacts.

05 TERMINAL

```
cat bulk_output/url.txt
```

EXPECT Offsets, URL features and context; may have no matches.

PURPOSE Examines web references.

06 TERMINAL

```
cat bulk_output/report.xml
```

EXPECT Run metadata and scanner information.

PURPOSE Documents the analysis process.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ bulk_extractor -o bulk_output forensic_evidence/evidence.dd
Output directory: bulk_output
... scanning ...
$ ls bulk_output
email.txt url.txt report.xml ...
```

HOW TO INTERPRET THE EVIDENCE

Bulk Extractor writes feature files, commonly with byte offsets and context. A match indicates strings in the evidence, not necessarily proof of a message sent or a website visited.

WRITE THIS AS YOUR FINAL OBSERVATION

Bulk Extractor was used to obtain and examine available email and URL artifacts from the image.

EXAM TIP

If the output folder exists from a previous attempt, choose a new name to avoid overwriting/conflicts. Feature files may be empty.

QUESTION 13 / EXPERIMENT 08 / WIRESHARK / Exp 8, pp. 1-5

Classify TCP, UDP, DNS and ICMP traffic

Analyze a network packet capture. Identify TCP, UDP, DNS, and ICMP traffic wherever available. Record the source and destination addresses, protocol types, and observations about the captured packets.

TARGET OUTCOME

Open/capture packets and classify four key protocols.

COMMAND-BY-COMMAND PLAYBOOK

01 AUTOPSY / WIRESHARK

```
Wireshark → File → Open → select .pcap/.pcapng OR capture on eth0
```

EXPECT Packet list appears.

PURPOSE Loads supplied packets or starts a controlled capture.

02 TERMINAL

```
ping -c 4 www.google.com
```

EXPECT ICMP echo requests and replies, if connectivity allows.

PURPOSE Optional: creates ping traffic during a fresh capture.

03 TERMINAL

```
nslookup google.com
```

EXPECT DNS query/reply text; actual resolver address varies.

PURPOSE Optional: creates DNS traffic during a fresh capture.

04 WIRESHARK FILTER

```
tcp
```

EXPECT Displays TCP packet records.

PURPOSE Wireshark display filter, not a shell command.

05 WIRESHARK FILTER

```
udp
```

EXPECT Displays UDP datagrams.

PURPOSE Wireshark display filter.

06 WIRESHARK FILTER

```
dns
```

EXPECT Displays DNS queries and responses where captured.

PURPOSE Wireshark display filter.

07 WIRESHARK FILTER

```
icmp
```

EXPECT Displays ICMP requests/replies where captured.

PURPOSE Wireshark display filter.

ILLUSTRATIVE CONSOLE / FINDINGS

```
Protocol | Source → Destination | Example observation
TCP | client → server | SYN/ACK or data
UDP | client → server | datagram transfer
DNS | client → resolver | A/AAAA query/response
ICMP | client ↔ destination | echo request/reply
```

HOW TO INTERPRET THE EVIDENCE

Record real source IP, destination IP, protocol, ports and packet Info from the capture. ICMP proves messaging between endpoints; DNS carries name queries; not every capture includes all protocols.

WRITE THIS AS YOUR FINAL OBSERVATION

The captured packets were classified by protocol and their source/destination details recorded.

EXAM TIP

Protocol names go into Wireshark display filter bar, not into the terminal. Do not invent absent packets.

QUESTION 14 / EXPERIMENT 05 / AUTOPSY GUI / Exp 5, pp. 3-7

Autopsy metadata, deleted files and keyword hits

Analyze a forensic disk image using Autopsy. Examine file metadata and investigate available deleted files, keyword matches, or other relevant artifacts. Document the evidence identified.

TARGET OUTCOME

Review file metadata and search or recovered-file artifacts.

COMMAND-BY-COMMAND PLAYBOOK

01 AUTOPSY / WIRESHARK

```
Autopsy → New/Open Case
```

EXPECT Case workspace appears.

PURPOSE Opens the investigation.

02 AUTOPSY / WIRESHARK

```
Add Data Source → Disk Image or VM File
```

EXPECT Supplied disk image becomes available.

PURPOSE Loads the correct evidence type.

03 AUTOPSY / WIRESHARK

```
Configure Ingest: File Type Identification + Keyword Search
```

EXPECT Autopsy indexes and categorizes supported evidence.

PURPOSE Enables relevant examination features.

04 AUTOPSY / WIRESHARK

```
Data Sources → select file → File Metadata
```

EXPECT Path, size, metadata times and allocation state.

PURPOSE Captures file properties.

05 AUTOPSY / WIRESHARK

```
Deleted Files → select available entry
```

EXPECT Deleted/unallocated artifact information, when recoverable.

PURPOSE Inspects removed material.

06 AUTOPSY / WIRESHARK

```
Keyword Hits → select matching item
```

EXPECT Hit term, preview and source path.

PURPOSE Documents keyword findings.

ILLUSTRATIVE CONSOLE / FINDINGS

```
Exp 5 example:
Recovered PDF: f0000000_Microsoft_Word_EHF_EXP_1_1.pdf
Size: 2,983,912 bytes | Type: application/pdf
Keyword "user": 3,572 results
```

HOW TO INTERPRET THE EVIDENCE

Allocation state and metadata require careful interpretation. Keyword counts reflect matching content, not a direct conclusion about intent or wrongdoing.

WRITE THIS AS YOUR FINAL OBSERVATION

Autopsy provided file metadata, available deleted-file data and keyword-search results for evidence review.

EXAM TIP

Report only artifacts actually present in the assigned image; example counts belong to Exp 5.

QUESTION 15 / EXPERIMENT 10 / EMAIL FORENSICS / Exp 10, pp. 1-5

Investigate email origin and key headers

Investigate a supplied email message using suitable Kali Linux tools. Extract and examine the sender, recipient, subject, date, message identifier, and return-path information. Record your observations.

TARGET OUTCOME

Preserve a local email copy and locate specified headers.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
mkdir -p email_forensics/{evidence,analysis,reports}
```

EXPECT Creates evidence, analysis and reports folders.

PURPOSE Prepares case workspace.

02 TERMINAL

```
cp ~/Downloads/suspicious_email.eml email_forensics/evidence/
```

EXPECT Copies supplied .eml into evidence folder.

PURPOSE Preserves a working evidence copy.

03 TERMINAL

```
cd email_forensics
```

EXPECT Enters investigation folder.

PURPOSE Makes relative paths work.

04 TERMINAL

```
sha256sum evidence/suspicious_email.eml
```

EXPECT SHA-256 digest of email evidence.

PURPOSE Records integrity reference.

05 TERMINAL

```
grep -i "^From:" evidence/suspicious_email.eml
```

EXPECT From: sender name/email.

PURPOSE Extracts visible sender.

06 TERMINAL

```
grep -i "^To:" evidence/suspicious_email.eml
```

EXPECT To: recipient details.

PURPOSE Extracts recipient.

07 TERMINAL

```
grep -i "^Subject:" evidence/suspicious_email.eml
```

EXPECT Subject line.

PURPOSE Extracts subject.

08 TERMINAL

```
grep -i "^Date:" evidence/suspicious_email.eml
```

EXPECT Date header.

PURPOSE Extracts claimed send date/time.

09 TERMINAL

```
grep -i "^Message-ID:" evidence/suspicious_email.eml
```

EXPECT Message-ID: <...>

PURPOSE Extracts email message identifier.

10 TERMINAL

```
grep -i "^Return-Path:" evidence/suspicious_email.eml
```

EXPECT Envelope return address, if present.

PURPOSE Checks return-path identity.

ILLUSTRATIVE CONSOLE / FINDINGS

```
Subject: Holiday Notice (02.10.2026 - Gandhi Jayanti)
Date: Thu, 1 Oct 2026 15:47:24 +0530
From: ...
To: ...
Message-ID: <...>
Return-Path: <...>
```

HOW TO INTERPRET THE EVIDENCE

Headers describe asserted origin, destination and identifiers. Compare From and Return-Path when assessing legitimacy, but mismatches can have innocent explanations.

WRITE THIS AS YOUR FINAL OBSERVATION

The sender, recipient, subject, timestamp, message identifier and return-path were extracted and recorded from the supplied email.

EXAM TIP

Exp 10 uses suspicious_email.eml. Replace the file location if needed. Multi-line folded headers may require reading the full raw message with less.

QUESTION 16 / EXPERIMENT 07 / THE SLEUTH KIT / Exp 7, pp. 6-10

List image files, inspect inodes and extract

Investigate a forensic disk image using The Sleuth Kit. List the available files and directories, examine the metadata of a designated file, or extract its contents as appropriate. Record and interpret the results.

TARGET OUTCOME

List file-system entries, inspect a selected inode, and attempt file recovery.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
fls -o 2048 evidence/data.dd
```

EXPECT Root directory entries with inode numbers.

PURPOSE Lists files/dirs from example partition.

02 TERMINAL

```
fls -o 2048 -r evidence/data.dd
```

EXPECT Recursive file and directory list.

PURPOSE Finds target file and its inode.

03 TERMINAL

```
istat -o 2048 evidence/data.dd INODE_NUMBER
```

EXPECT Allocation, size, timestamps and inode metadata.

PURPOSE Examines the designated file.

04 TERMINAL

```
icat -o 2048 evidence/data.dd INODE_NUMBER > recovered_file
```

EXPECT Saves file bytes when recoverable; may report missing blocks.

PURPOSE Extracts available content.

05 TERMINAL

```
file recovered_file
```

EXPECT Recognizes file format when nonempty and complete.

PURPOSE Checks the extracted output.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ fls -o 2048 evidence/data.dd
d/d 5505025: home
d/d 2883585: etc
...
$ istat -o 2048 evidence/data.dd INODE_NUMBER
Inode / Size / Timestamps / Allocation ...
```

HOW TO INTERPRET THE EVIDENCE

fls prints file and inode records, istat provides metadata, and icat attempts to reconstruct selected content. In Exp 7, extraction from the incomplete image failed for an attempted inode.

WRITE THIS AS YOUR FINAL OBSERVATION

The file-system entries and available metadata were examined; extraction succeeds only if the relevant data blocks are present.

EXAM TIP

Use the inode from fls and the correct partition offset from mmls. Do not claim success if the image is truncated.

QUESTION 17 / EXPERIMENT 10 / EMAIL FORENSICS / Exp 10, pp. 6-13

Trace email routing and authentication

Examine the routing and authentication information in a supplied email message. Analyze the available SPF, DKIM, and DMARC results and explain how these findings help assess email authenticity.

TARGET OUTCOME

Read Received and Authentication-Results traces in the supplied message.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
grep -i "^Received:" evidence/suspicious_email.eml
```

EXPECT Visible Received hop lines; long lines may fold.

PURPOSE Examines mail-handling path.

02 TERMINAL

```
grep -i "spf" evidence/suspicious_email.eml
```

EXPECT SPF pass/fail or related policy references.

PURPOSE Locates SPF evaluation.

03 TERMINAL

```
grep -i "dkim" evidence/suspicious_email.eml
```

EXPECT DKIM signature and pass/fail references.

PURPOSE Locates DKIM evaluation.

04 TERMINAL

```
grep -i "dmarc" evidence/suspicious_email.eml
```

EXPECT DMARC result and policy, if present.

PURPOSE Locates domain-alignment evaluation.

05 TERMINAL

```
grep -i "Authentication-Results:" evidence/suspicious_email.eml
```

EXPECT Combined authentication-result header line(s).

PURPOSE Cross-checks receiving server results.

ILLUSTRATIVE CONSOLE / FINDINGS

```
Received: from mail-sor-f69.google.com ([209.85.220.69]) ...
spf=pass
dkim=pass
dmarc=pass
p=NONE
```

HOW TO INTERPRET THE EVIDENCE

SPF pass indicates authorized sending IP per evaluated domain; DKIM pass indicates a valid signature; DMARC pass indicates domain alignment via SPF or DKIM. p=none is generally a monitoring policy, not message rejection.

WRITE THIS AS YOUR FINAL OBSERVATION

Routing headers and SPF, DKIM, and DMARC results were reviewed to assess technical authenticity signals, without concluding that a passing email is harmless.

EXAM TIP

Run after Q15 setup or first cd into email_forensics. grep may miss continuation lines; open the raw email in less for full context.

QUESTION 18 / EXPERIMENT 01 / LINUX ESSENTIALS / Exp 1, pp. 2-5

Create, rename, display and delete files

Create and manage a directory structure in Kali Linux. Perform file creation, copying, renaming, displaying, and deletion operations. Verify the final directory contents and document the procedure.

TARGET OUTCOME

Demonstrate the complete file lifecycle while retaining the renamed copy.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
mkdir EHF_Files
```

EXPECT Directory created.

PURPOSE Creates workspace.

02 TERMINAL

```
cd EHF_Files
```

EXPECT Enters workspace.

PURPOSE Sets correct location.

03 TERMINAL

```
echo "Digital Evidence" > original.txt
```

EXPECT Creates written sample.

PURPOSE Creates evidence text file.

04 TERMINAL

```
cp original.txt copy.txt
```

EXPECT Creates duplicate.

PURPOSE Copies content.

05 TERMINAL

```
mv copy.txt renamed.txt
```

EXPECT New name appears.

PURPOSE Renames the copy.

06 TERMINAL

```
cat renamed.txt
```

EXPECT Digital Evidence

PURPOSE Displays stored content.

07 TERMINAL

```
rm original.txt
```

EXPECT Original removed.

PURPOSE Deletes only requested file.

08 TERMINAL

```
ls -l
```

EXPECT Only renamed.txt among created files.

PURPOSE Verifies final outcome.

ILLUSTRATIVE CONSOLE / FINDINGS

```
$ cat renamed.txt
Digital Evidence
$ ls -l
-rw-r--r-- ... renamed.txt
```

HOW TO INTERPRET THE EVIDENCE

Only renamed.txt remains. Avoid rm -rf because the task only requests ordinary file deletion.

WRITE THIS AS YOUR FINAL OBSERVATION

Directory creation, file creation, copying, renaming, displaying and deletion were carried out, and the final folder was verified.

EXAM TIP

Unlike Q1, this question deletes the original and keeps the renamed copy.

QUESTION 19 / EXPERIMENT 08 / WIRESHARK / Exp 8, pp. 3-5

Analyze DNS exchange or TCP handshake

Analyze a supplied packet capture or capture traffic during controlled lab communication. Identify DNS queries and responses or TCP handshake packets wherever available. Explain the observed network communication.

TARGET OUTCOME

Identify name resolution or three-stage TCP connection establishment.

COMMAND-BY-COMMAND PLAYBOOK

01 AUTOPSY / WIRESHARK

```
Wireshark → Open supplied capture OR start controlled capture
```

EXPECT Packet pane receives or displays traffic.

PURPOSE Loads communications to analyze.

02 TERMINAL

```
nslookup google.com
```

EXPECT DNS query and returned addresses if resolution works.

PURPOSE Optional controlled DNS traffic generator.

03 TERMINAL

```
curl https://example.com
```

EXPECT HTTPS request that commonly uses TCP/TLS.

PURPOSE Optional controlled TCP traffic generator.

04 WIRESHARK FILTER

```
dns
```

EXPECT Shows query and response records, if available.

PURPOSE Wireshark DNS display filter.

05 WIRESHARK FILTER

```
tcp
```

EXPECT Shows TCP segments, including SYN/SYN-ACK/ACK where captured.

PURPOSE Wireshark TCP display filter.

ILLUSTRATIVE CONSOLE / FINDINGS

```
DNS: query A google.com → response A <resolved IP>
TCP handshake:
1. Client → Server : SYN
2. Server → Client : SYN, ACK
3. Client → Server : ACK
```

HOW TO INTERPRET THE EVIDENCE

DNS translates domain names into records. In a TCP handshake, SYN initiates, SYN-ACK acknowledges and replies, and ACK completes the setup. All three must belong to the same connection.

WRITE THIS AS YOUR FINAL OBSERVATION

The available DNS records and/or TCP connection-setup packets were identified and their sequence explained.

EXAM TIP

Do not infer a full three-way handshake if the capture starts midway or uses another transport. Wireshark filters do not belong in a Linux shell.

QUESTION 20 / EXPERIMENT 06 / BULK EXTRACTOR / Exp 6, pp. 2-4

Extract emails, URLs and IP artifacts

Analyze a forensic disk image to identify network-related artifacts, including available email addresses, URLs, and IP addresses. Examine the generated output files and document your findings.

TARGET OUTCOME

Collect all three feature types using Bulk Extractor and inspect output report.

COMMAND-BY-COMMAND PLAYBOOK

01 TERMINAL

```
ls -lh forensic_evidence/evidence.dd
```

EXPECT Evidence image path and size.

PURPOSE Checks supplied input.

02 TERMINAL

```
bulk_extractor -o bulk_output_q20 forensic_evidence/evidence.dd
```

EXPECT Scans image and writes output.

PURPOSE Produces independent results directory for Q20.

03 TERMINAL

```
ls bulk_output_q20
```

EXPECT Output feature files, including email/url/ip if generated.

PURPOSE Shows analysis products.

04 TERMINAL

```
cat bulk_output_q20/email.txt
```

EXPECT Offsets/email features or only header.

PURPOSE Examines email matches.

05 TERMINAL

```
cat bulk_output_q20/url.txt
```

EXPECT Offsets/URL features or only header.

PURPOSE Examines URL matches.

06 TERMINAL

```
cat bulk_output_q20/ip.txt
```

EXPECT Offsets/IP-like features, if file exists.

PURPOSE Examines available IP results.

07 TERMINAL

```
cat bulk_output_q20/report.xml
```

EXPECT Scanner and run information.

PURPOSE Documents process.

ILLUSTRATIVE CONSOLE / FINDINGS

```
email.txt → user@example.com at byte offset ...
url.txt → https://example.com at byte offset ...
ip.txt → 192.168.1.10 at byte offset ...
```

HOW TO INTERPRET THE EVIDENCE

Matches are extracted byte patterns; verify context because addresses can be incidental or false positives and do not alone prove actual network activity.

WRITE THIS AS YOUR FINAL OBSERVATION

Available email, URL and IP-like features were identified and documented from the supplied forensic image.

EXAM TIP

Use a fresh output folder to avoid collision with Q12. If ip.txt is absent or empty, state that explicitly.

FINAL REVISION / 03

One-minute command map

Read the task verb first, then select the minimum tool needed.

Create / copy / rename / remove	mkdir · touch / echo · cp · mv · rm · ls
Identify system + interface	uname -a · cat /etc/os-release · date · history · ip a
Domain registration / technologies	whois · whatweb
Public hosts / DNS / emails	theHarvester · dnsenum · dmitry
Open ports / service versions	nmap · nmap -sV
Create and hash logical image	tar · dd · ls -lh · sha256sum
Autopsy artifacts	New Case → Data Source → Ingest → Metadata / Deleted Files / Hits
Bulk artifacts	bulk_extractor → email.txt / url.txt / ip.txt
Disk layout and file system	mmls → fsstat
Files / inodes / extraction	fls → istat → icat
Packet analysis	Wireshark → tcp / udp / dns / icmp filters
Email header analysis	grep From / To / Subject / Date / Message-ID / Return-Path / Received

HOW TO WRITE OUTPUTS HONESTLY

The examples in this book are illustrative. Where a tool returns no match, a DNS lookup fails, a disk image is partial, or an Autopsy artifact is unavailable, the correct practical response is to document that outcome - not to invent a successful result.

END OF HANDBOOK • OPEN THE COMPANION HTML FOR PRACTICE MODE, FLASH QUESTIONS AND PROGRESS TRACKING.